PostgreSQL One-to-Many Relationships

5 Combined Medium–Hard Interview Practice Questions • Intermediate Level (70–75%)

Practice Rules

Focus on JOINs, GROUP BY, HAVING, aggregate functions, CASE, WHERE, ORDER BY, DISTINCT, and practical one-to-many relationship logic. Do not use subqueries, CTEs, or window functions for these questions.

Relationship Model

departments (1) → employees (many)
employees (1) → projects (many)
customers (1) → orders (many)
products (1) → order_items (many)

1. Departments & Employees — Performance Summary

Write a query that returns one row per department with: department name, total employees, average salary rounded to 2 decimals, highest salary, lowest salary, and the number of employees whose performance_rating is 8 or higher. Show only departments having at least 3 employees. Sort by average salary descending.

departments

department_id	department_name	location
1	Engineering	Mumbai
2	Data Science	Pune
3	HR	Mumbai
4	Finance	Pune
5	Marketing	Bengaluru
6	Operations	Hyderabad

employees

employee_id	employee_name	department_id	salary	performance_rating
101	Aarav	1	85000	9
102	Diya	1	72000	7
103	Kabir	1	95000	10
104	Meera	1	68000	8
105	Rohan	2	90000	9
106	Anaya	2	78000	8
107	Ishaan	2	82000	7
108	Sara	3	62000	9
109	Vikram	3	58000	6
110	Nisha	4	88000	8
111	Arjun	4	76000	7
112	Tara	5	70000	8
113	Neel	5	65000	6
114	Kavya	6	60000	7
115	Om	6	64000	8

2. Department Salary & High-Performer Analysis

Return department name, employee count, total salary expense, average performance rating (rounded to 2 decimals), and a column named performance_status. Set it to 'Strong' when the department has at least 2 employees with performance_rating >= 8, otherwise 'Needs Review'. Return only departments where total salary expense is greater than 200000. Sort by total salary expense descending.

Use the same **departments** and **employees** tables above.

3. Customers & Orders — Revenue Classification

Write a query that returns every customer who has placed at least 2 orders. Display customer name, number of orders, total order amount, average order amount rounded to 2 decimals, and a customer_type column. Classify customers as 'VIP' when total order amount >= 100000, 'Regular' when total order amount is between 50000 and 99999.99, and 'Basic' otherwise. Sort by total order amount descending.

customers

customer_id	customer_name	city
201	Aditya	Mumbai
202	Sneha	Pune
203	Rahul	Nashik
204	Ira	Mumbai
205	Manav	Nagpur
206	Riya	Pune

orders

order_id	customer_id	order_date	total_amount	status
301	201	2026-01-05	45000	Delivered
302	201	2026-02-10	62000	Delivered
303	202	2026-01-12	28000	Delivered
304	202	2026-02-18	35000	Shipped
305	202	2026-03-01	22000	Delivered
306	203	2026-01-20	55000	Cancelled
307	203	2026-02-22	30000	Delivered
308	204	2026-01-25	70000	Delivered
309	205	2026-02-02	15000	Delivered
310	205	2026-03-04	18000	Delivered
311	206	2026-03-10	60000	Shipped
312	206	2026-03-21	50000	Delivered

4. Customers, Orders & Order Status Analysis

Return every customer, including customers who have no orders. Display customer name, total number of orders, delivered orders, cancelled orders, total value of delivered orders, and an activity_status column. Set activity_status to 'Active' if the customer has at least 2 delivered orders, 'At Risk' if they have exactly 1 delivered order, and 'No Purchase' if they have no delivered orders. Sort Active customers first, then by delivered order value descending.

Use the **customers** and **orders** tables above.

5. Product Sales — One-to-Many + Multi-Table Challenge

Using products, orders, and order_items, return one row per product with product name, category, number of distinct orders containing the product, total quantity sold, total sales amount, and a sales_level column. Classify products as 'High' when total sales >= 100000, 'Medium' when total sales is between 50000 and 99999.99, and 'Low' otherwise. Include products that have never been ordered. Sort ordered products by total sales descending.

products

product_id	product_name	category	unit_price
401	Laptop Pro	Electronics	75000
402	Wireless Mouse	Accessories	1500
403	Mechanical Keyboard	Accessories	4500
404	Monitor 27	Electronics	22000
405	USB-C Hub	Accessories	3000
406	Office Chair	Furniture	12000
407	Webcam HD	Electronics	6000
408	Desk Lamp	Furniture	2500

order_items

order_item_id	order_id	product_id	quantity	unit_price
501	301	401	1	75000
502	301	402	2	1500
503	302	404	2	22000
504	302	403	3	4500
505	303	402	4	1500
506	303	405	2	3000
507	304	406	3	12000
508	305	407	5	6000
509	307	404	1	22000
510	308	401	1	75000
511	308	403	2	4500
512	309	402	3	1500
513	310	405	4	3000
514	311	406	2	12000
515	312	401	1	75000
516	312	407	2	6000
517	312	402	5	1500

Important: For Question 5, calculate sales using order_items.quantity × order_items.unit_price. Treat cancelled orders as excluded from sales.

Interview Skill Checklist

• INNER JOIN vs LEFT JOIN in one-to-many relationships

• GROUP BY at the correct parent level

• COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)

• SUM(), AVG(), MIN(), MAX() with grouped data

• HAVING for filtering grouped results

• CASE expressions for business classifications

• Conditional aggregation using CASE inside aggregate functions

• Handling parent records with zero child records

• Joining three tables without double-counting

• Sorting grouped results using aggregate expressions

Recommended order: Solve Q1 → Q2 → Q3 → Q4 → Q5 without looking at solutions. For interview practice, first write the JOIN structure, then decide the GROUP BY level, then add aggregates, HAVING/WHERE conditions, and finally CASE/ORDER BY.